

Traffic Light Recognition

1. Course Content

Note: This program runs on a sandbox map; you will need to adjust the program for your personal map.

1. Learn the actions the car takes after recognizing traffic lights
2. Learn the newly emerged key source code

2. Road Sign Detection

Road sign detection source code path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/scripts/auto_drive.py
```

3. Program Startup

3.1 Startup Command

- Parameter Configuration

Parameter path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/config/auto_drive_node.yaml
```

```
turn_radius: 0.4
linear_speed: 0.13
angular_speed: -0.55
duration: 2.0
response_distance_1: 2900
response_distance_2: 132
cooldown_time: 0.2
DEBUG_MODE: False
DEBUG_MODE_2: True
START_AutoDrive: True
Kp: -0.62
Ki: 0.0
Kd: -0.1
max_integral: 1000.0
image_size: [640, 480]
```

After modifying parameters, restart the launch file to use the modified parameters.

- Start camera + chassis + autonomous driving node (need to run in virtual environment terminal)

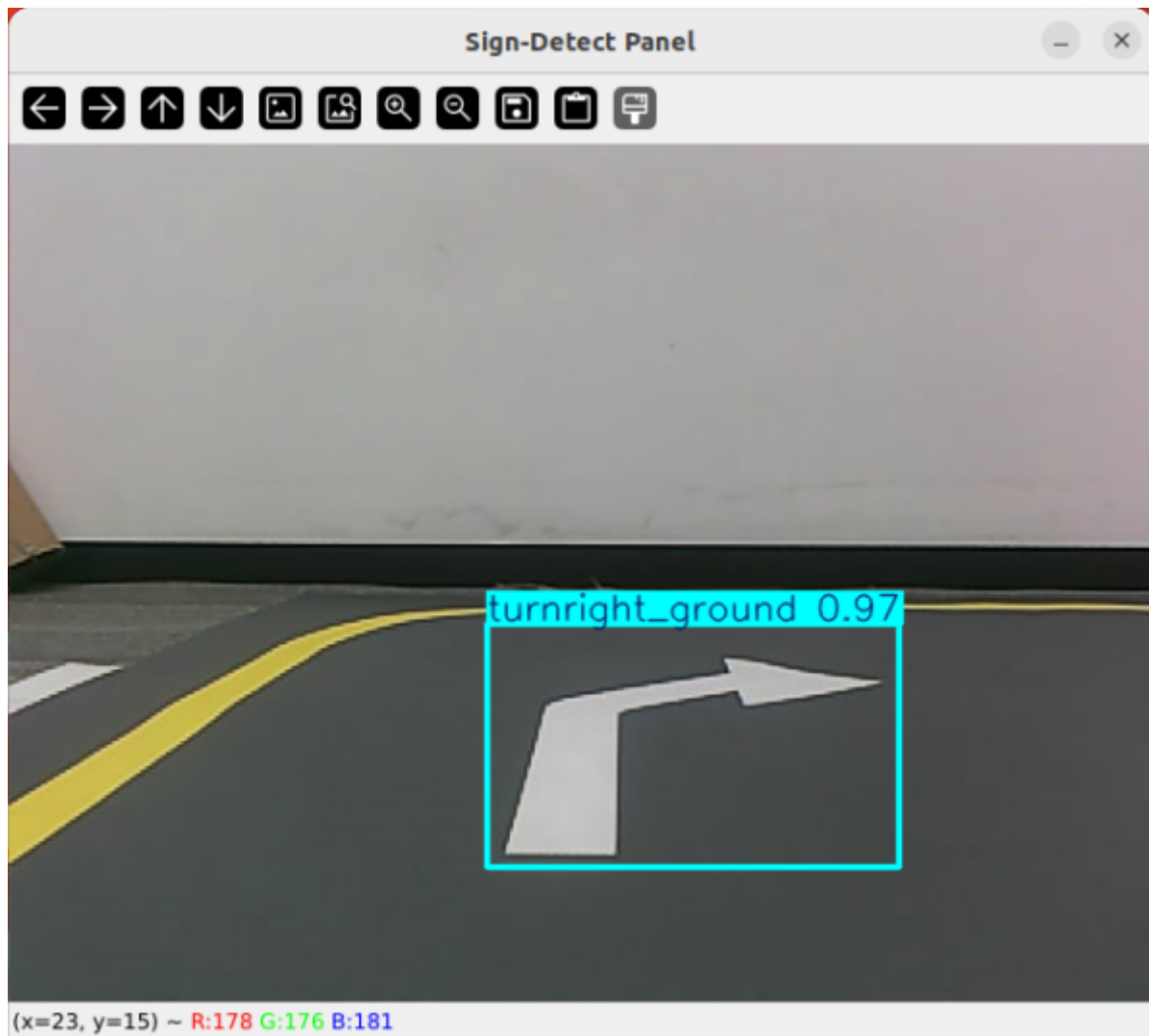
```
roslaunch auto_driver auto_drive.launch
```

```
(yolov11) jetson@yahboom:~$ roslaunch auto_driver auto_driver.launch
... logging to /home/jetson/.ros/log/2add7f36-d0c0-11f0-9d82-48b02d5ab21e/roslaunch-yahboom-15809.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://yahboom:44059/

SUMMARY
=====
PARAMETERS
* /ascamera_hp60c/color_pcl: False
* /ascamera_hp60c/confiPath: /home/jetson/yahb...
* /ascamera_hp60c/depth_height: 480
* /ascamera_hp60c/depth_width: 640
* /ascamera_hp60c/fps: 30
* /ascamera_hp60c/pub_mono8: False
* /ascamera_hp60c/pub_tfTree: True
* /ascamera_hp60c/rgb_height: 480
* /ascamera_hp60c/rgb_width: 640
* /ascamera_hp60c/usb_bus_no: -1
* /ascamera_hp60c/usb_path: null
* /auto_driver/DEBUG_MODE: False
* /auto_driver/DEBUG_MODE_2: True
* /auto_driver/Kd: -0.1
* /auto_driver/Ki: 0.0
* /auto_driver/Kp: -0.62
* /auto_driver/START_AutoDrive: False
* /auto_driver/angular_speed: -0.55
* /auto_driver/cooldown_time: 0.2
* /auto_driver/duration: 2.0
* /auto_driver/image_size: [640, 480]
* /auto_driver/linear_speed: 0.13
* /auto_driver/max_integral: 1000.0
* /auto_driver/response_distance_1: 2900
* /auto_driver/response_distance_2: 132
* /auto_driver/turn_radius: 0.4
* /auto_driver/turn_speed: 0.4
```

After successful startup, the display will not move to test road sign recognition. (If you want it to move, change the parameter START_AutoDrive in auto_drive_node.yaml to True)



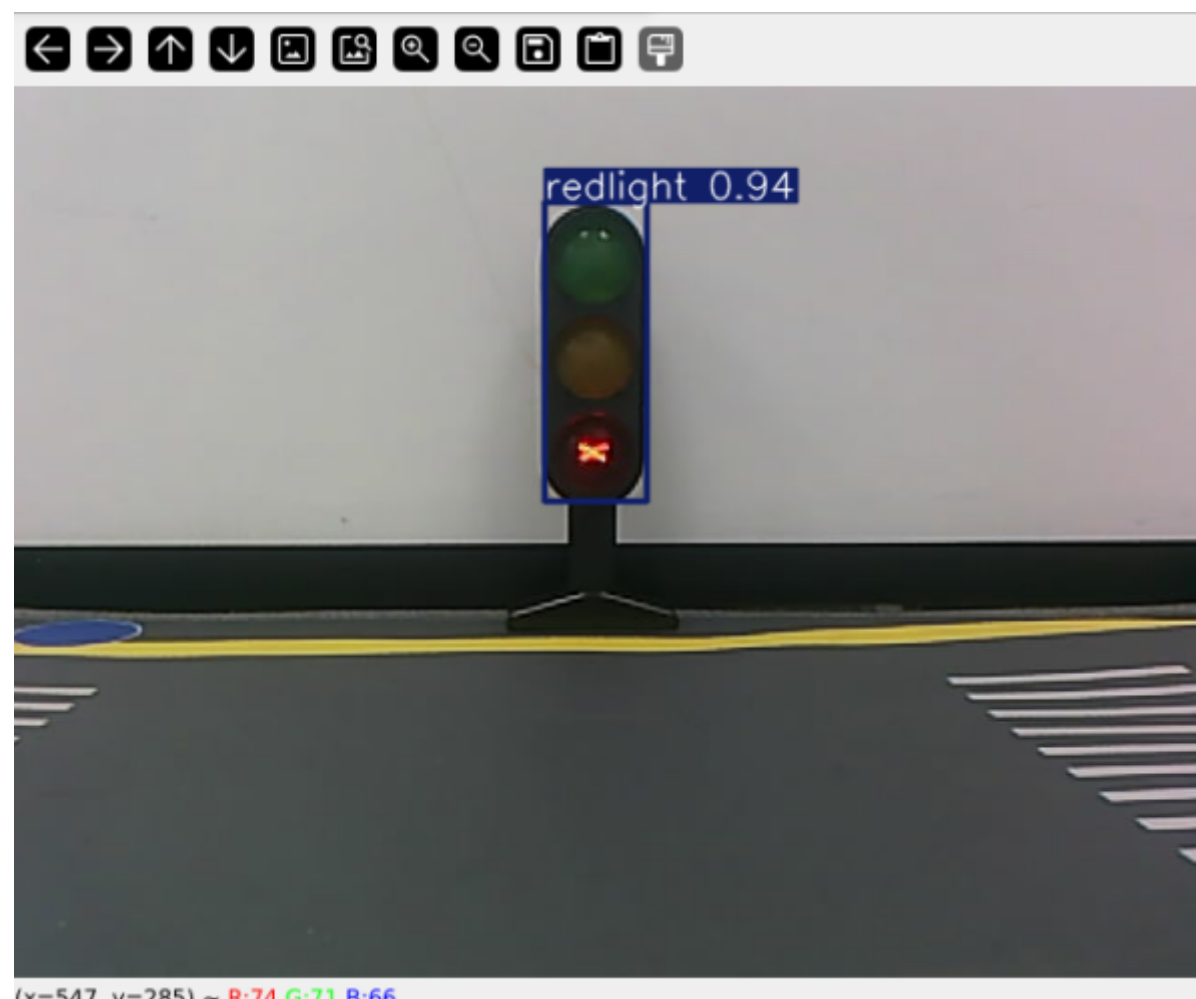
After the program runs, it will display the YOLO recognition screen. The car will run centered on the lane lines and perform corresponding actions based on the road signs recognized during driving.

3.2 Car movement after recognizing traffic lights

When the program recognizes red light, the program will print

```
jetson@yahboom: ~  
jetson@yahboom: ~ 80x24  
[auto_drive-4] standardized_midline_points start_point:(360, 477) end_point:(348  
, 357),midline_length:120  
[auto_drive-4] Loading /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_dr  
ive/tools/lane_V6.engine for TensorRT inference...  
[auto_drive-4] [12/02/2025-18:29:35] [TRT] [I] Loaded engine size: 11 MiB  
[yolo_detect-3] [12/02/2025-18:29:35] [TRT] [I] Loaded engine size: 12 MiB  
[auto_drive-4] [12/02/2025-18:29:35] [TRT] [W] Using an engine plan file across  
different models of devices is not recommended and is likely to affect performan  
ce or even cause errors.  
[yolo_detect-3] [12/02/2025-18:29:35] [TRT] [W] Using an engine plan file across  
different models of devices is not recommended and is likely to affect performa  
nce or even cause errors.  
[ascamera_node-1] [INFO] [1764671375.556312801] [ascamera_hp60c.camera_publisher  
]: 2025-12-02 18:29:35[INFO] [CameraHp60c.cpp] [1148] [streamCallback] set gain  
ret 0, gain 4  
[auto_drive-4] [12/02/2025-18:29:35] [TRT] [I] [MemUsageChange] TensorRT-managed  
allocation in IExecutionContext creation: CPU +0, GPU +19, now: CPU 0, GPU 28 (M  
iB)  
[yolo_detect-3] [12/02/2025-18:29:35] [TRT] [I] [MemUsageChange] TensorRT-manage  
d allocation in IExecutionContext creation: CPU +0, GPU +19, now: CPU 0, GPU 28  
(MiB)  
[auto_drive-4] [INFO] [1764671378.387579116] [auto_drive] current_sign redlight  
[auto_drive-4] [INFO] [1764671378.390984163] [auto_drive] red_stop
```

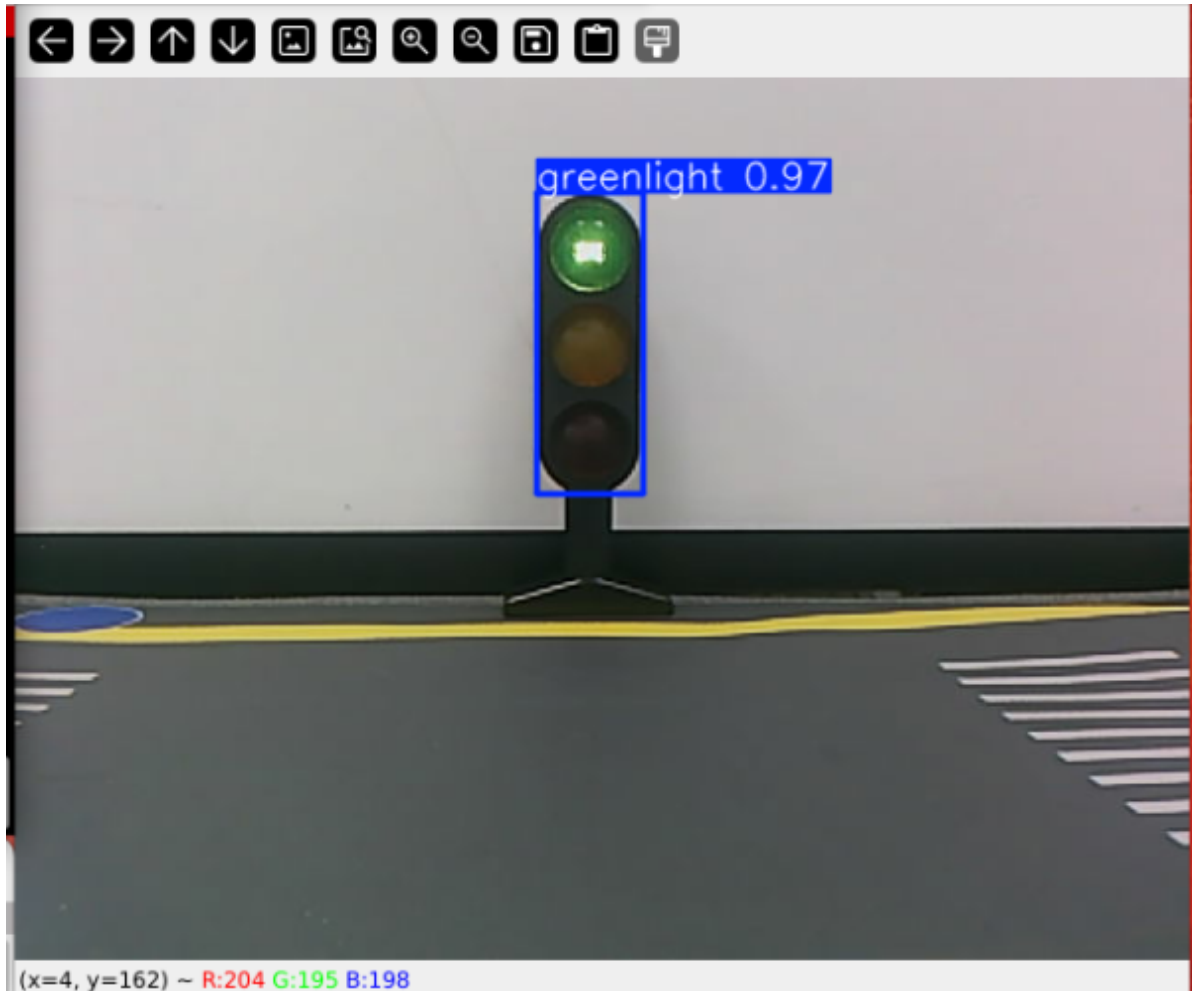
The car will keep stopping at red light and wait for green light



The red light handling function is in the auto_drive.py file. Below is the function that is entered after recognizing red light. Users can modify the recognized actions themselves

```
def red_stop(self):  
    self.flag = True  
    rospy.loginfo('red_stop')  
    self._set_current_speed(linear_x=0.0)
```

When the program recognizes green light, the program will print Go straight, the car releases the parking state and moves forward along the initial speed.



The green light handling function is in the auto_drive.py file. Below is the function that is entered after recognizing green light. Users can modify the car's movement after recognizing the road sign themselves

```
def go_straight(self):  
    rospy.loginfo('Go straight')  
    self.flag = False  
    self._set_current_speed(linear_x=self.drive_speed)
```

4. Source Code Analysis

4.1, auto_drive.py

YOLO signpost message reception:

```
results = self.sign_detect.get_infer_result(frame, True)
annotated_frame = frame.copy()
```

Multiple filtering mechanisms: mainly to prevent continuously recognizing road signs and triggering actions; the action will only be unlocked when different road signs are recognized.

```
# If a sign that requires cooldown is detected, start cooldown
if detected_sign_requires_cooldown:
    self.sign_cooldown = True
    self.last_sign_time = current_time
    rospy.loginfo(f"Sign detected, starting {self.cooldown_duration}s cooldown")
# Check if the repeated sign should be ignored
should_ignore = False
if obj.class_name == self.last_sign:
    should_ignore = True
```

Event queue processing

```
def _event_handling_worker(self):
    '''Event handling worker thread'''
    # sign handling mapping table
    sign_handlers = {
        "straight": self.go_straight,
        "turnright": lambda: self.turn('right'),
        "turnright_ground": self.do_nothing,
        "honking": self.car_horn,
        "sidewalk": self.slow_down,
        "sidewalk_ground": self.do_nothing,
        "speedlimit": self.slow_down,
        "stop": self.stop,
        "school": self.school,
        "parkA": self.parking_A,
        "parkB": self.parking_A,
        "greenlight": self.go_straight,
        "redlight": self.red_stop,
        "yellowlight": self.stop,
        "out_park": self.out_parking
    }

    # sign enable index mapping
    sign_enable_map = {
        "straight": 10, "turnright": 11, "honking": 1, "sidewalk": 6,
        "sidewalk_ground": 7, "speedlimit": 8, "stop": 9, "school": 5,
        "parkA": 2, "parkB": 3, "greenlight": 0, "redlight": 4,
        "yellowlight": 13
    }
```

Specific action implementation

```
def red_stop(self):
    self.flag = True
    rospy.loginfo('red_stop')
    self._set_current_speed(linear_x=0.0)

def go_straight(self):
    rospy.loginfo('Go straight')
    self.flag = False
    self._set_current_speed(linear_x=self.drive_speed)
```